

How slam2.0 works

A stereo visual SLAM system, from one camera frame to a closed loop — the geometry, the optimizer, the code, and the bugs that taught us the most.

KITTI odometry, sequence 00 · 4541 stereo frames · 3.7 km

Architecture after ORB-SLAM2 / ORB-SLAM3; estimator written from scratch in NumPy

Companion to the source tree at `src/slam/`

1.73 m

ATE, full sequence

0.88 %

translation drift

0

lost frames

4

loop closures, all true

Contents

1	What problem this solves	3
2	The papers, and what we took from each	5
3	Geometry you need: camera, stereo, SE(3)	7
4	The data structures the whole system stands on	10
5	Frame in: ORB features and stereo matching	13
6	Frame 0: starting a map from nothing	17
7	Frame 1 and after: tracking	18
8	The optimizer, derived	25
9	Local mapping: growing and cleaning the map	31
10	Loop closing: recognising a place and fixing the past	37
11	Three threads and one lock	45
12	Measuring it: ATE, RPE, and what the numbers mean	47
13	Three bugs worth knowing by heart	49
14	What is missing, and what I would build next	50
A	Appendix A — file map	51
B	Appendix B — glossary	52

How to read this. Chapters 3 to 10 follow a single camera frame through the system in the order the code actually runs. Every listing is pulled out of the repository when this document is built, so what you read here is what runs. Boxes marked *Why this way* explain a design choice you will be asked about; boxes marked *Bug story* are real failures from this project — they are the most useful interview material in the document. Each chapter ends with questions you should be able to answer out loud.

1

What problem this solves

A camera on a moving car produces images. We want two things out of them: where the camera is at every instant, and a map of the world it is looking at. Doing both at once, without a prior map and without GPS, is Simultaneous Localization and Mapping.

1.1 Odometry drifts, and drift compounds

The naive approach is visual odometry: match features between consecutive frames, compute the motion between them, and chain the motions together. Each step carries a small error. Chaining multiplies those errors together, so position error grows without bound — and the error is in the *pose*, which means it also rotates everything you estimate afterwards. On a 3.7 km sequence a 1% drift rate is tens of metres by the end.

SLAM fixes this by keeping a map and using it. If the camera sees a wall it already has points for, and those points are still in the map with good positions, the pose can be computed *against the map* rather than against the previous frame. The error then does not compound: it is bounded by how well the map is known in the area you are in.

1.2 Three kinds of data association

ORB-SLAM3's paper makes this the organising idea of the whole field, and it is the clearest way to remember what the parts of this system do:

ASSOCIATION	WHAT IT MEANS	WHERE IT LIVES HERE
Short-term	Match to the last few frames. Pure odometry. Always drifts.	<code>Tracker._search_last</code> — the constant-velocity step
Mid-term	Match to map elements that are still geometrically close, i.e. the accumulated drift is small enough that projection works. This is what kills drift.	<code>Tracker._track_local_map</code> and local bundle adjustment
Long-term	Recognise a place regardless of drift, using appearance. Fixes the global shape of the trajectory.	<code>LoopCloser</code> — bag of words, then a pose graph

Why this way

A system with only short-term association is a visual odometry system, whatever it calls itself. The single most important sentence in this document: **the map must feed back into pose estimation**. An earlier version of this project built a perfectly good map and never used it for tracking — it looked like SLAM from the outside and drifted like odometry, because it was odometry.

1.3 Why stereo

A single camera cannot tell a small nearby object from a large distant one: monocular SLAM recovers the world only up to an unknown scale factor, and that scale itself drifts. Two cameras a known distance apart (the *baseline*,

53.7 cm on the KITTI car) fix the scale from the first frame. That one fact removes an entire class of problems: no scale drift, no need for a delicate initialisation procedure, and loop closure becomes a rigid-body correction instead of a 7-degree-of-freedom similarity correction.

Check yourself

1. Why does matching against a map bound the error, when matching against the previous frame does not?
2. Which of the three associations would you lose if you deleted the bag-of-words vocabulary? What would the trajectory look like on a sequence that revisits a street?
3. Name the two things stereo gives you that monocular does not.

2

The papers, and what we took from each

This system is not a new algorithm. It is a careful implementation of a well-understood one, and every component has a source. Being able to say which idea came from where — and why it is there — is most of what “I built this” means.

PAPER	THE IDEA WE USED	WHERE IT LIVES
ORB-SLAM Mur-Artal, Montiel, Tardós, TRO 2015	Keyframe SLAM rather than filtering; the same ORB features for tracking, mapping and place recognition; the covisibility graph; the essential graph for pose-graph optimisation; generous keyframe insertion followed by culling.	map/structure.py, tracking.py
ORB-SLAM2 Mur-Artal, Tardós, TRO 2017	Stereo keypoints (u , v , u_R); the close/far split at 35 baselines; map initialisation from the very first stereo frame; SE(3) loop closing; local BA with fixed outer keyframes.	features/orb.py, mapping.py
ORB-SLAM3 Campos et al., TRO 2021	The short/mid/long-term framing used in Chapter 1; place recognition that scores candidates over covisible groups instead of single keyframes.	loop.py
DBow2 Gálvez-López, Tardós, TRO 2012	A vocabulary tree over binary descriptors, TF-IDF weighting, L1 scoring, and an inverted file so querying does not touch every keyframe.	features/vocab.py
Bundle Adjustment — A Modern Synthesis Triggs et al., 2000	The Schur complement trick that makes BA tractable, and robust kernels in the normal equations.	optimize/solver.py
Closed-form absolute orientation Horn, 1987	Least-squares rigid transform between two sets of 3-D points, in closed form.	geometry/align.py — loop verification and ATE alignment
Multiple View Geometry Hartley & Zisserman	DLT triangulation, the fundamental matrix, epipolar constraints.	geometry/triangulate.py, mapping.py
KITTI benchmark Geiger et al., 2012	The dataset, and the relative-drift metric over 100–800 m segments.	eval.py, data/kitti.py
TUM RGB-D benchmark Sturm et al., 2012	Absolute trajectory error as RMSE after rigid alignment.	eval.py

2.1 What we deliberately did not take

- **Inertial fusion** (ORB-SLAM3's headline contribution). There is no IMU in our input stream.
- **The Atlas / multi-map system.** Worth having when tracking fails; tracking has not failed on our sequence, so building it would be untested code.
- **Monocular initialisation** (the homography/fundamental model selection). Stereo makes it unnecessary.
- **Sim(3) loop closing.** Stereo fixes scale, so the 7-DoF similarity collapses to a 6-DoF rigid transform — exactly the simplification ORB-SLAM2 makes over ORB-SLAM.

3

Geometry you need: camera, stereo, SE(3)

Two pieces of mathematics carry the entire system: how a 3-D point becomes pixels, and how to represent and update a camera pose. Everything else is bookkeeping around them.

3.1 The pinhole camera

A point in camera coordinates $X_c = (X, Y, Z)$ projects to pixels by

$$u = f_x X / Z + c_x, \quad v = f_y Y / Z + c_y \quad (3.1)$$

with focal lengths f_x, f_y in pixels and principal point (c_x, c_y) . On KITTI sequence 00, $f_x = 718.856$ px and the images are 1241×376 .

3.2 The stereo keypoint

The two KITTI cameras are rectified: they have the same intrinsics and are displaced along the x axis by the baseline $b = 0.537$ m, so the same world point lands on the *same image row* in both. Its horizontal position in the right image is

$$u_R = f_x (X - b) / Z + c_x \quad (3.2)$$

so a measurement is the triple (u, v, u_R) — ORB-SLAM2 calls this a *stereo keypoint*. The difference $d = u - u_R$ is the disparity, and inverting the equation gives depth directly:

$$Z = f_x b / d \quad (3.3)$$

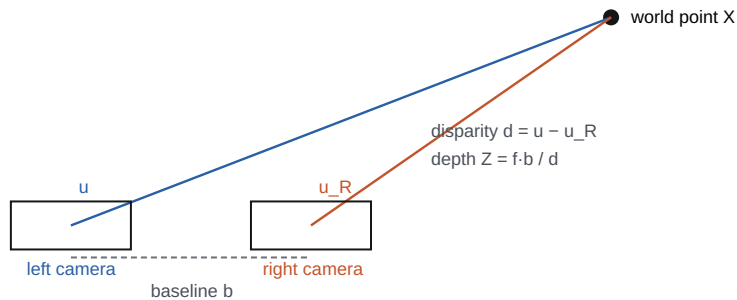


Figure 3.1. Stereo in one picture. Disparity is large for near points and shrinks to zero at infinity, so depth uncertainty grows as Z^2 — the reason ORB-SLAM2 treats "close" and "far" points differently.

Both live in one small class, and the projection used by the optimizer is the stereo one:

src/slam/data/frame.py

```
def project_stereo(self, Xc: np.ndarray) -> np.ndarray:
    uv = self.project(Xc)
    Xc = np.asarray(Xc, dtype=np.float64).reshape(-1, 3)
    z = np.maximum(Xc[:, 2], 1e-12)
    ur = self.fx * (Xc[:, 0] - self.baseline) / z + self.cx
    return np.column_stack([uv, ur])
```

Why this way

Because depth error grows with Z^2 , a point at 5 m is a precise measurement and a point at 80 m is barely more than a bearing. ORB-SLAM2's answer, which we copy, is the **close/far split** at $Z < 35b \approx 18.8$ m: close points are trusted enough to create map points immediately from one frame, far points only through triangulation across several views, and the keyframe rule watches the count of close points (Chapter 7).

3.3 Poses live on a manifold, not in a vector space

A camera pose is a rotation and a translation. We store it as a 4×4 matrix taking world coordinates to camera coordinates,

$$T_{cw} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}, \quad X_c = R X_w + t \quad (3.4)$$

The subtlety: the set of valid rotations is not a vector space. You cannot optimise by adding a correction to R , because $R + \Delta$ is generally not a rotation any more. The standard fix is to parameterise *updates* in the Lie algebra: a six-vector $\xi = (\omega, v)$ — three for rotation, three for translation — mapped onto the manifold by the exponential map, and applied on the left:

$$T \leftarrow \exp(\xi^\wedge) T \quad (3.5)$$

The optimizer therefore always works with a small six-vector, and the pose is guaranteed to stay a valid pose because $\exp()$ returns a rotation by construction. Our implementation is the textbook Rodrigues formula:

src/slam/geometry/lie.py

```
def exp_se3(xi: np.ndarray) -> np.ndarray:
    xi = np.asarray(xi, dtype=np.float64).reshape(6)
    w, v = xi[:3], xi[3:]
    T = np.eye(4)
    T[:3, :3] = exp_so3(w)
    T[:3, 3] = _v_matrix(w) @ v
    return T
```

and the inverse map, used to turn a relative pose into a twist:

src/slam/geometry/lie.py

```
def log_se3(T: np.ndarray) -> np.ndarray:
    T = np.asarray(T, dtype=np.float64)
    w = log_so3(T[:3, :3])
    v = _v_inv_matrix(w) @ T[:3, 3]
    return np.concatenate([w, v])
```

Bug story — poses that stopped being poses

Floating-point round-off means a pose that has been multiplied a few thousand times is no longer exactly orthonormal. That is normally harmless. It was not harmless here, because the constant-velocity model computes $T @ \text{invert}(T_{\text{prev}})$, and $\text{invert}()$ uses R^T as R^{-1} — which is only true for an exact rotation. Feed a slightly-off matrix in, get a worse one out, use it to predict, repeat. The error **tripled every frame**: 3×10^{-14} at frame 5, 10^{-6} at frame 25, 5×10^{-2} at frame 37, tracking dead at frame 41.

Two fixes, both kept. The velocity is now stored as a twist, so prediction goes through $\exp()$ and cannot leave the manifold. And every estimator projects its input poses back onto $SO(3)$ first — which is exactly what g2o gets for free by storing rotations as quaternions and re-normalising.

src/slam/geometry/lie.py

```
def orthonormalize(T: np.ndarray) -> np.ndarray:
    """Project the rotation block onto S0(3) (nearest rotation, via SVD)."""
    U, _, Vt = np.linalg.svd(np.asarray(T, dtype=np.float64)[:3, :3])
    out = np.array(T, dtype=np.float64)
    out[:3, :3] = U @ np.diag([1.0, 1.0, np.linalg.det(U @ Vt)]) @ Vt
    return out
```

Check yourself

1. A point is measured with disparity 2 px at $f_x = 719$, $b = 0.54$ m. How far away is it, and how much does the depth change if the disparity is off by half a pixel?
2. Why do we apply pose updates on the left ($\exp(\xi) T$) rather than adding to the matrix entries?
3. What does $\text{invert}()$ assume, and what happens when that assumption is slightly violated a few hundred times in a row?

4

The data structures the whole system stands on

Four objects. Everything the system does is an operation on these, so it is worth knowing them precisely before following the execution flow.

4.1 Frame, MapPoint, KeyFrame, Map

OBJECT	WHAT IT IS
Frame	One stereo image pair plus what we extracted from it: keypoints, descriptors, octaves, <code>u_right</code> , and a pose. Cheap and discarded after use.
MapPoint	A 3-D point in the world: position, the keyframes that observe it and at which keypoint index, a representative descriptor, a mean viewing direction, a valid distance range, and the counters that decide whether it survives culling.
KeyFrame	A frame promoted to permanent status: its pose, its features, its map points, plus its links — covisibility weights, a spanning-tree parent, and loop edges.
Map	The set of keyframes and points, a lock, and a version counter.

4.2 Two derived quantities that do a lot of work

Every map point maintains a **mean viewing direction** and a **scale-invariance distance range**. They exist so that a later frame can ask a cheap question — “could I even see this point from here, and at what pyramid level would it appear?” — without trying to match it:

src/slam/map/structure.py

```
def update_normal_and_depth(self) -> None:
    if not self.obs or self.ref_kf not in self.obs:
        return
    v = self.Xw - np.stack([kf.center for kf in self.obs])
    n = (v / np.linalg.norm(v, axis=1, keepdims=True)).mean(axis=0)
    self.normal = n / np.linalg.norm(n)
    ref = self.ref_kf
    self.dmax = np.linalg.norm(self.Xw - ref.center) * SCALES[ref.octave[self.obs[ref]]]
    self.dmin = self.dmax / SCALES[-1]
```

The range comes from the octave the point was detected at: a point found at pyramid level l at distance d would be detectable between roughly $d \cdot 1.2^l$ and that divided by 1.2^7 . Inverting that relation predicts the level at which it should appear now, which tells the matcher which octaves to search:

src/slam/features/orb.py

```
def predict_level(dmax: np.ndarray, dist: np.ndarray) -> np.ndarray:
    """Pyramid level at which a point with scale-invariance distance dmax appears at `dist`."""
    return np.clip(np.ceil(np.log(dmax / np.maximum(dist, 1e-12)) / np.log(SCALE)), 0, N_LEVELS - 1).astype(int)
```

Both feed the frustum test, which is used identically by tracking, by fusion and by loop closing:

src/slam/map/structure.py

```
def visible(Xw, normal, dmin, dmax, T_cw, cam):
    """ORB-SLAM's frustum test: in front, inside the image, within the scale-invariance
    distance range, and seen from under 60 degrees. Returns the visible points'
    (indices, pixels, viewing cosine, predicted pyramid level)."""
    Xc = transform_points(T_cw, Xw)
    P0 = Xw - invert(T_cw)[:3, 3]
    dist = np.linalg.norm(P0, axis=1)
    view = np.einsum("ij,ij->i", P0, normal) / dist
    uv = cam.project(Xc)
    ok = (Xc[:, 2] > 0) & cam.in_image(uv) & (dist >= 0.8 * dmin) & (dist <= 1.2 * dmax) & (view >= 0.5)
    idx = np.flatnonzero(ok)
    return idx, uv[idx], view[idx], predict_level(dmax[idx], dist[idx])
```

4.3 The covisibility graph and the spanning tree

Two keyframes are *covisible* if they observe the same map points; the edge weight is how many. This graph is the backbone of the system:

- **Tracking** uses it to decide which part of the map to project into the current frame.
- **Local BA** optimises exactly the covisible neighbourhood of the newest keyframe.
- **Loop closing** uses it to group candidates, and optimises the *essential graph*: the spanning tree, plus edges with weight ≥ 100 , plus loop edges.

src/slam/map/structure.py

```

def update_connections(self) -> None:
    counts: dict[KeyFrame, int] = {}
    for _, mp in self.valid_points():
        for kf in mp.obs:
            if kf is not self and not kf.bad:
                counts[kf] = counts.get(kf, 0) + 1
    if not counts:
        return
    strong = {kf: w for kf, w in counts.items() if w >= MIN_COVIS}
    if not strong:
        kf = max(counts, key=counts.get)
        strong = {kf: counts[kf]}
    for kf, w in strong.items():
        kf._connect(self, w)
    self.covis = strong
    self._reorder()
    if self.parent is None and self is not self.map.origin:
        self.change_parent(self.ordered[0])

```

Why this way

The full covisibility graph is dense — thousands of keyframes, each linked to dozens. Optimising all of it after a loop closure would be needlessly slow, and optimising only the spanning tree would be too weak. The essential graph is ORB-SLAM's compromise: keep the tree so the graph stays connected, keep only strong edges, add the loop. That is the difference between a pose graph that solves in a second and one that does not.

Check yourself

1. Why does a map point store a viewing direction at all — what goes wrong without it when the car drives back down the same street in the opposite direction?
2. A keyframe is culled. What happens to its spanning-tree children, and why must the culled keyframe remember T_{cp} ?

5

Frame in: ORB features and stereo matching

The pipeline starts here. Everything downstream sees only keypoints and descriptors — the images are never looked at again.

5.1 ORB, and why features are spread on purpose

ORB gives a corner position, a pyramid level (octave), an orientation, and a 256-bit binary descriptor that can be compared with an XOR and a bit count. We ask OpenCV for twice the budget and then keep the strongest per grid cell, because a natural detector piles features onto whatever is most textured — a tree, a fence — and leaves the rest of the image empty. Spread features mean better-conditioned geometry:

src/slam/features/orb.py

```
def _bucket(pts: np.ndarray, resp: np.ndarray, n: int, cell: int) -> np.ndarray:
    if len(pts) <= n:
        return np.arange(len(pts))
    cid = (pts[:, 1] // cell).astype(int) * 10000 + (pts[:, 0] // cell).astype(int)
    order = np.lexsort((-resp, cid))
    c = cid[order]
    start = np.flatnonzero(np.r_[True, c[1:] != c[:-1]])
    rank = np.arange(len(c)) - np.repeat(start, np.diff(np.r_[start, len(c)]))
    cand = order[rank < int(np.ceil(1.5 * n / len(start)))]
    return cand[np.argsort(-resp[cand])[:n]]
```

5.2 Matching left to right

Because the pair is rectified, the search for a keypoint's stereo partner is one-dimensional: same row, to the left, within a disparity range. Three filters do the work — a row band scaled by the octave, an octave-similarity test, and a disparity range — and only then do we compare descriptors:

src/slam/features/orb.py

```
def match_stereo(frame: Frame, pr: np.ndarray, dr: np.ndarray, octr: np.ndarray) -> None:
    """ORB-SLAM2 ComputeStereoMatches: row-band descriptor match, then SAD subpixel refinement."""
    n = len(frame.keypoints)
    frame.u_right = np.full(n, np.nan)
    if n == 0 or len(pr) == 0:
        return
    order = np.argsort(pr[:, 1])
    pr, dr, octr = pr[order], dr[order], octr[order]

    ul, vl, octl = frame.keypoints[:, 0], frame.keypoints[:, 1], frame.octave
    band = 2.0 * SCALES[-1]
    i, j = ragged_pairs(np.searchsorted(pr[:, 1], vl - band), np.searchsorted(pr[:, 1], vl + band,
    "right"))
```

```

disp = ul[i] - pr[j, 0]
ok = (
    (np.abs(vl[i] - pr[j, 1]) <= 2.0 * SCALES[octr[j]])
    & (np.abs(octl[i] - octr[j]) <= 1)
    & (disp >= 0)
    & (disp <= frame.camera.fx)
)
i, j = i[ok], j[ok]
li, k, best, _ = best_two(i, hamming(frame.descriptors[i], dr[j]))
good = best < (TH_HIGH + TH_LOW) // 2
li, rj = li[good], j[k[good]]

ur, sad = _subpixel(frame.image_left, frame.image_right, frame.keypoints[li], pr[rj, 0],
octl[li])
d = ul[li] - ur
keep = np.isfinite(ur) & (d >= 0) & (d < frame.camera.fx)
ur = np.where(d <= 0, ul[li] - 0.01, ur)
if keep.any():
    keep &= sad <= 1.5 * 1.4 * np.median(sad[keep])
frame.u_right[li[keep]] = ur[keep]

```

The descriptor match gives an integer pixel. That is not accurate enough: one pixel of disparity error at 20 m is about 1.4 m of depth. So each match is refined by a sliding sum-of-absolute-differences window on the keypoint's own pyramid level, followed by a parabola fit through the best three scores — giving a subpixel `u_right`:

src/slam/features/orb.py

```

def _subpixel(left, right, kl, ur0, octv):
    """1D SAD search on the keypoint's pyramid level plus a parabola fit."""
    ur = np.full(len(kl), np.nan)
    best = np.full(len(kl), np.inf)
    off = np.arange(-_W, _W + 1)
    shifts = np.arange(-_L, _L + 1)
    h0, w0 = left.shape
    for lvl in np.unique(octv):
        s = SCALES[lvl]
        size = (round(w0 / s), round(h0 / s))
        IL = left if lvl == 0 else cv2.resize(left, size, interpolation=cv2.INTER_LINEAR)
        IR = right if lvl == 0 else cv2.resize(right, size, interpolation=cv2.INTER_LINEAR)
        h, w = IL.shape
        m = np.flatnonzero(octv == lvl)
        cx = np.round(kl[m, 0] / s).astype(int)
        cy = np.round(kl[m, 1] / s).astype(int)
        cr = np.round(ur0[m] / s).astype(int)
        inb = (cy >= _W) & (cy + _W < h) & (cx >= _W) & (cx + _W < w) & (cr >= _L + _W) & (cr + _L +
_W < w)
        m, cx, cy, cr = m[inb], cx[inb], cy[inb], cr[inb]
        if len(m) == 0:

```

```

        continue

    rows = cy[:, None, None] + off[None, :, None]
    pl = IL[rows, cx[:, None, None] + off[None, None, :]].astype(np.float32)
    pl -= pl[:, _W : _W + 1, _W : _W + 1]
    cols = cr[:, None, None, None] + shifts[None, :, None, None] + off[None, None, None, :]
    pr = IR[rows[:, None], cols].astype(np.float32)
    pr -= pr[:, :, _W : _W + 1, _W : _W + 1]
    sad = np.abs(pl[:, None] - pr).sum(axis=(2, 3))
    b = sad.argmin(axis=1)
    a = np.arange(len(m))
    bi = np.clip(b, 1, 2 * _L - 1)
    d1, d2, d3 = sad[a, bi - 1], sad[a, bi], sad[a, bi + 1]
    den = 2.0 * (d1 + d3 - 2.0 * d2)
    delta = np.divide(d1 - d3, den, out=np.zeros_like(den), where=den != 0)
    ok = (b > 0) & (b < 2 * _L) & (np.abs(delta) <= 1.0)
    ur[m[ok]] = s * (cr[ok] + shifts[b[ok]] + delta[ok])
    best[m[ok]] = sad[a, b][ok]

return ur, best

```

Why this way

Notice the shape of that code: no Python loop over keypoints. Patches for every match at a given pyramid level are gathered with one fancy-index operation into an array of shape (matches, shifts, 11, 11), and the SAD scores for all of them are one `abs().sum()`. This is the pattern used everywhere in this codebase — the loop is over the eight pyramid levels, not over the two thousand features. It is the reason a pure-NumPy SLAM system runs at all.

5.3 The primitives everything else matches with

Three small functions in `features/matcher.py` are reused by tracking, mapping and loop closing. First, Hamming distance over whole arrays via a 256-entry popcount table:

`src/slam/features/matcher.py`

```

def hamming(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Hamming distance over the last axis of broadcastable uint8 descriptor arrays."""
    return _POP[np.bitwise_xor(a, b)].sum(axis=-1, dtype=np.int32)

```

Second, a way to turn "for each item, a variable-length list of candidates" into flat index arrays — the idiom that replaces nested loops throughout:

src/slam/features/matcher.py

```
def ragged_pairs(lo: np.ndarray, hi: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """All (i, j) with lo[i] <= j < hi[i], as flat index arrays."""
    cnt = np.maximum(hi - lo, 0)
    i = np.repeat(np.arange(len(lo)), cnt)
    j = np.arange(cnt.sum()) - np.repeat(np.cumsum(cnt) - cnt, cnt) + np.repeat(lo, cnt)
    return i, j
```

Third, the mutual-best-match rule, with an optional Lowe-style ratio test:

src/slam/features/matcher.py

```
def one_to_one(p: np.ndarray, q: np.ndarray, d: np.ndarray, max_dist: int, ratio: float | None = None):
    """Mutually closest (p, q) pairs: best q per p (optional ratio test), then best p per q."""
    g, k, best, second = best_two(p, d)
    keep = best <= max_dist
    if ratio is not None:
        keep &= best < ratio * second
    p, q, d = g[keep], q[k[keep]], best[keep]
    qj, k2, _, _ = best_two(q, d)
    return p[k2], qj, d[k2]
```

Check yourself

1. Why search for the stereo partner only to the *left* of the keypoint, and only within $d \in [0, f_x]$?
2. What would break if we skipped subpixel refinement — in the map, and in the estimated speed of the car?
3. Explain `ragged_pairs` to someone who has never seen it. What are the two output arrays, and what replaces the inner loop?

6

Frame 0: starting a map from nothing

Monocular SLAM needs a careful two-view initialisation. Stereo does not: the first frame already contains metric depth, so the map exists after one image pair.

src/slam/tracking.py

```
def _initialize(self, frame: Frame) -> None:
    stereo = np.flatnonzero(stereo_depth(frame) > 0)
    if len(stereo) < self.cfg.min_init_points:
        return
    frame.T_cw = np.eye(4)
    kf = self.map.new_keyframe(frame)
    for i, X in zip(stereo, unproject(frame, stereo, np.eye(4))):
        frame.points[i] = self._new_point(X, kf, i)
    self.mapper.insert(kf)
    self.ref_kf = self.last_kf = kf
    self.local_kfs = [kf]
    self.state = "ok"
```

Every keypoint with a valid stereo match is unprojected into the world and becomes a map point; the frame becomes keyframe 0, which is also the origin of the world frame and the one pose that is never optimised — it fixes the gauge (see Chapter 8). On KITTI 00 this yields about 1150 points immediately.

The unprojection is the stereo equations of Chapter 3, applied to an index array:

src/slam/tracking.py

```
def unproject(frame, idx: np.ndarray, T_wc: np.ndarray) -> np.ndarray:
    kp = frame.keypoints[idx]
    Xc = frame.camera.triangulate_stereo(kp[:, 0], kp[:, 1], frame.u_right[idx])
    return Xc @ T_wc[:3, :3].T + T_wc[:3, 3]
```

7

Frame 1 and after: tracking

This is the per-frame hot path, and it runs in the order below. Read `Tracker._track` as the table of contents for this chapter.

src/slam/tracking.py

```
def _track(self, frame: Frame) -> None:
    self._update_last_frame()
    ok = False
    if self.state == "ok" and self.velocity is not None and frame.frame_id >= self.last_reloc +
2:
        ok = self._track_motion_model(frame)
    if not ok:
        ok = self._track_wide(frame)
    ok = ok and self._track_local_map(frame)
    if not ok:
        self.state = "lost"
        frame.T_cw = self._predict()
        return
    if self.state == "lost":
        self.last_reloc = frame.frame_id
    self.state = "ok"
    # Kept as a twist: exp() of it is always a rotation, so repeated prediction cannot
    # walk the pose off SE(3) (composing T @ invert(T_prev) does, ~3x per frame).
    self.velocity = log_se3(frame.T_cw @ invert(self.last.T_cw))
    if self._need_keyframe(frame):
        self._create_keyframe(frame)
    frame.points[frame.outlier] = None
```

7.1 Step 1 — predict

The cheapest useful prior is that the camera keeps doing what it was doing. We store the last frame-to-frame motion as a twist and apply it to the last pose:

src/slam/tracking.py

```
def _predict(self) -> np.ndarray:
    if self.velocity is None:
        return self.last.T_cw.copy()
    return exp_se3(self.velocity) @ self.last.T_cw
```

Before that, the last frame's pose is re-anchored on its reference keyframe. This matters: bundle adjustment or a loop closure may have moved that keyframe since, and the velocity model must be measured against the corrected world, not a stale one.

src/slam/tracking.py

```
def _update_last_frame(self) -> None:
    # Re-anchor on the reference keyframe: BA or a loop closure may have moved it since.
    self.last.T_cw = _through_ref(*self.traj[-1])
    pts = self.last.points
    for i, mp in enumerate(pts):
        while mp is not None and mp.replaced is not None:
            mp = mp.replaced
        pts[i] = None if mp is None or mp.bad else mp
```

7.2 Step 2 — find the last frame's points in this image

Each map point seen in the last frame is projected into the predicted pose and searched for in a window around its projection. Two details make this reliable. The search radius scales with the octave, because a feature detected at a coarse level is intrinsically less well localised. And the range of octaves searched depends on the direction of travel: driving forward, a point's scale can only grow, so we search from its last octave upward.

src/slam/tracking.py

```
def _search_last(self, frame: Frame, th: float) -> int:
    """Project the last frame's map points; octave window follows the motion direction."""
    frame.points[:] = None
    last = self.last
    idx = np.flatnonzero(np.not_equal(last.points, None))
    if len(idx) == 0:
        return 0
    mps = last.points[idx]
    tz = (last.T_cw @ invert(frame.T_cw))[2, 3]
    b = frame.camera.baseline
    o = last.octave[idx]
    lo = np.where(tz > b, o, np.where(-tz > b, 0, o - 1))
    hi = np.where(tz > b, N_LEVELS - 1, np.where(-tz > b, o, o + 1))
    Xw = np.stack([mp.Xw for mp in mps])
    desc = np.stack([mp.descriptor for mp in mps])
    p, j, _ = project_search(frame, Xw, desc, th * SCALES[o], lo, hi, TH_HIGH)
    keep = rotation_consistent(last.angle[idx[p]], frame.angle[j])
    frame.points[j[keep]] = mps[p[keep]]
    return int(keep.sum())
```

The matches are then filtered by an orientation-consistency histogram — genuine matches share a common rotation between the two frames, so matches outside the three dominant bins are discarded:

src/slam/features/matcher.py

```
def rotation_consistent(a_src: np.ndarray, a_dst: np.ndarray, bins: int = 30) -> np.ndarray:
    """ORB-SLAM orientation check: keep matches in the three dominant rotation bins."""
    b = np.round(((a_src - a_dst) % 360.0) * bins / 360.0).astype(int) % bins
    cnt = np.bincount(b, minlength=bins)
    top = np.argsort(-cnt)[:3]
    top = top[cnt[top] >= 0.1 * cnt[top[0]]]
    return np.isin(b, top)
```

7.3 Step 3 — solve for the pose

With 3-D points fixed and 2-D/3-D measurements in hand, the pose is the solution of a small non-linear least squares problem — **motion-only bundle adjustment**, derived in Chapter 8. In code it is one call plus bookkeeping of which observations turned out to be outliers:

src/slam/tracking.py

```
def _optimize_pose(self, frame: Frame, count_found: bool = False) -> int:
    idx = np.flatnonzero(np.not_equal(frame.points, None))
    frame.outlier[:] = False
    if len(idx) < 3:
        return 0
    X = np.stack([mp.Xw for mp in frame.points[idx]])
    z = np.column_stack([frame.keypoints[idx], frame.u_right[idx]])
    frame.T_cw, inl = motion_only_ba(frame.T_cw, X, z, INV_SIGMA2[frame.octave[idx]],
    frame.camera)
    frame.outlier[idx[~inl]] = True
    if count_found:
        for mp in frame.points[idx[inl]]:
            mp.n_found += 1
    else:
        frame.points[idx[~inl]] = None
    return int(inl.sum())
```

7.4 Step 4 — track the local map

Matching only the last frame's points is still short-term association, so drift would still accumulate. The fix is to pull in the part of the map that should be visible *now*. K1 is the set of keyframes that share points with the current frame; K2 adds one covisible neighbour, one child and the parent of each, which extends the window without exploding it:

src/slam/tracking.py

```

def _update_local_keyframes(self, frame: Frame) -> None:
    """K1 = keyframes sharing points with the frame; K2 = one covisible, child, parent each."""
    counts: dict[KeyFrame, int] = {}
    for i, mp in enumerate(frame.points):
        if mp is None:
            continue
        if mp.bad:
            frame.points[i] = None
            continue
        for kf in mp.obs:
            counts[kf] = counts.get(kf, 0) + 1
    k1 = [kf for kf in sorted(counts, key=counts.get, reverse=True) if not kf.bad]
    if not k1:
        return
    fid = frame.frame_id
    for kf in k1:
        kf.track_ref = fid
    local = list(k1)
    for kf in k1:
        if len(local) > 80:
            break
        for group in (kf.best_covisibles(10), kf.children, [kf.parent] if kf.parent else []):
            for n in group:
                if not n.bad and n.track_ref != fid:
                    n.track_ref = fid
                    local.append(n)
                    break
    self.local_kfs = local
    self.ref_kf = k1[0]

```

All points of those keyframes that are not already matched are put through the frustum test of Chapter 4, and the survivors are searched for with a radius that depends on the viewing angle — a point seen straight-on is predicted more precisely than one seen obliquely:

src/slam/tracking.py

```

def _search_local_points(self, frame: Frame, local: list[MapPoint]) -> None:
    fid = frame.frame_id
    for mp in frame.points:
        if mp is not None:
            mp.n_visible += 1
            mp.last_frame = fid
    cand = [mp for mp in local if mp.last_frame != fid and not mp.bad]
    if not cand:
        return
    Xw, normal, dmin, dmax, desc = point_arrays(cand)
    idx, _, view, level = visible(Xw, normal, dmin, dmax, frame.T_cw, frame.camera)
    for k in idx:
        cand[k].n_visible += 1
    th = 3.0 if fid < self.last_reloc + 2 else 1.0
    radius = th * SCALES[level] * np.where(view > 0.998, 2.5, 4.0)
    p, j, _ = project_search(frame, Xw[idx], desc[idx], radius, level - 1, level, TH_HIGH, ratio=0.8)
    for a, b in zip(p, j):
        frame.points[b] = cand[idx[a]]

```

Then the pose is optimised a second time, now against several hundred map points from many keyframes. **This second optimisation is the mid-term association that keeps the system from drifting**, and it is what separates this from visual odometry.

7.5 Step 5 — decide whether this frame becomes a keyframe

Keyframes are inserted generously and culled later. The conditions are ORB-SLAM2's, including the stereo one that matters on a road: if the number of tracked *close* points falls below 100 while more than 70 untracked close points are available, the camera is moving into new nearby structure and needs a keyframe regardless of how well tracking is going.

src/slam/tracking.py

```
def _need_keyframe(self, frame: Frame) -> bool:
    n_kfs = len(self.map.keyframes)
    if frame.frame_id < self.last_reloc + self.cfg.max_frames and n_kfs > self.cfg.max_frames:
        return False
    ref_matches = self.ref_kf.tracked_points(3 if n_kfs > 2 else 2)
    idle = self.mapper.idle()

    depth = stereo_depth(frame)
    close = depth < self.cfg.th_depth * frame.camera.baseline
    tracked = np.not_equal(frame.points, None) & ~frame.outlier
    need_close = (close & tracked).sum() < 100 and (close & ~tracked).sum() > 70

    since = frame.frame_id - self.last_kf.frame_id
    c1 = since >= self.cfg.max_frames or idle
    c1 = c1 or self.n_inliers < 0.25 * ref_matches or need_close
    th_ref = 0.75 if n_kfs >= 2 else 0.4
    c2 = (self.n_inliers < th_ref * ref_matches or need_close) and self.n_inliers > 15
    if not (c1 and c2):
        return False
    if idle:
        return True
    self.mapper.interrupt_ba()
    return self.mapper.queue_len() < 3
```

Creating one also creates map points from the closest stereo keypoints — nearest first, always the close ones, and at least a hundred:

src/slam/tracking.py

```

def _create_keyframe(self, frame: Frame) -> None:
    """New keyframe plus map points from its closest stereo keypoints (>= 100, all close
    ones)."""
    kf = self.map.new_keyframe(frame)
    depth = stereo_depth(frame)
    cand = np.flatnonzero(depth > 0)
    cand = cand[np.argsort(depth[cand])]
    new = [i for i in cand if frame.points[i] is None]
    X = dict(zip(new, unproject(frame, np.array(new, dtype=int), kf.T_wc))) if new else {}
    th = self.cfg.th_depth * frame.camera.baseline
    for n, i in enumerate(cand, 1):
        if frame.points[i] is None:
            frame.points[i] = self._new_point(X[i], kf, i)
            if depth[i] > th and n > 100:
                break
    self.ref_kf = self.last_kf = kf
    self.mapper.insert(kf)

```

Why this way

Every frame stores its pose *relative to its reference keyframe*, not as an absolute pose. When bundle adjustment or a loop closure moves the keyframes, the whole trajectory moves with them for free. Storing absolute per-frame poses — which an earlier version of this project did — means the trajectory you print is the one before all your optimisations, and none of the corrections ever show up in the output.

src/slam/tracking.py

```

def trajectory(self) -> list[np.ndarray]:
    """Per-frame T_cw re-expressed through the (optimized) reference keyframes."""
    return [_through_ref(ref, T_cr) for ref, T_cr in self.traj]

```

Check yourself

1. Tracking succeeds against the last frame but the local-map step then reports 28 inliers. What happens to this frame, and what is the state afterwards?
2. Why does the octave window in `_search_last` depend on `tz`, the forward component of the motion?
3. The car stops at a traffic light for 200 frames. Which keyframe condition prevents the map from growing pointlessly, and which one would fire if a bus pulled up alongside?

8

The optimizer, derived

Both bundle adjustment problems in this system minimise the same thing: the disagreement between where map points *should* appear and where they were measured. This chapter derives it once, in the form the code implements.

8.1 The residual

For an observation of point X_w in the camera with pose T_{cw} , measured at the stereo keypoint $z = (u, v, u_R)$:

$$r = \pi_s(T_{cw} X_w) - z \in \mathbb{R}^3 \quad (8.6)$$

and we minimise the weighted sum of squares over all observations, with a robust kernel ρ :

$$\min \sum_k \rho(r_k^T \Sigma_k^{-1} r_k) \quad (8.7)$$

The information Σ^{-1} is a scalar per observation: a keypoint found at octave l is localised to about 1.2^l pixels, so its weight is $1/1.2^{2l}$. A monocular keypoint — no stereo match — simply has its third residual row zeroed, which is how one code path serves both cases:

src/slam/optimize/solver.py

```
def _project(Xc: np.ndarray, z: np.ndarray, cam: StereoCalib):
    x, y, d = Xc.T
    iz = 1.0 / np.where(np.abs(d) > 1e-6, d, 1e-6)
    stereo = ~np.isnan(z[:, 2])
    pred = np.stack(
        [cam.fx * x * iz + cam.cx, cam.fy * y * iz + cam.cy, cam.fx * (x - cam.baseline) * iz +
        cam.cx],
        axis=1,
    )
    r = pred - z
    J = np.zeros((len(Xc), 3, 3))
    J[:, 0, 0] = J[:, 2, 0] = cam.fx * iz
    J[:, 1, 1] = cam.fy * iz
    J[:, 0, 2] = -cam.fx * x * iz**2
    J[:, 1, 2] = -cam.fy * y * iz**2
    J[:, 2, 2] = -cam.fx * (x - cam.baseline) * iz**2
    r[~stereo, 2] = 0.0
    J[~stereo, 2] = 0.0
    return r, J, stereo
```

8.2 The Jacobians

Differentiating the projection with respect to the camera-frame point gives the 3×3 block visible in the listing above:

$$\frac{\partial \pi_s}{\partial X_c} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \\ \frac{f_x}{Z} & 0 & -\frac{f_x(X-b)}{Z^2} \end{bmatrix} \quad (8.8)$$

For the pose we need the derivative of the point with respect to a left perturbation. Using $\exp(\xi) \approx I + \xi^\wedge$ for small $\xi = (\omega, v)$:

$$\exp(\xi^\wedge) X_c \approx X_c + \omega \times X_c + v \implies \partial/\partial \xi = [-[X_c]_\times \mid I] \quad (8.9)$$

src/slam/optimize/solver.py

```
def _dxi(Xc: np.ndarray) -> np.ndarray:
    """d(exp(xi) Xc)/d(xi) at xi = 0, i.e. [-[Xc]x | I], batched to (K, 3, 6)."""
    x, y, z = Xc.T
    D = np.zeros((len(Xc), 3, 6))
    D[:, 0, 1], D[:, 0, 2] = z, -y
    D[:, 1, 0], D[:, 1, 2] = -z, x
    D[:, 2, 0], D[:, 2, 1] = y, -x
    D[:, _D3, _D3 + 3] = 1.0
    return D
```

and with respect to the point itself, $\partial X_c / \partial X_w = R$. Chaining gives the two Jacobian blocks the solver assembles: $J_{pose} = (\partial \pi / \partial X_c) \cdot [-[X_c]_\times \mid I]$ and $J_{point} = (\partial \pi / \partial X_c) \cdot R$.

8.3 Robustness: Huber and χ^2

Wrong matches are inevitable, and a squared cost lets one bad match at 40 px dominate a hundred good ones at 1 px. The Huber kernel makes the cost linear beyond a threshold δ , which in iteratively reweighted least squares is just a per-observation weight $\delta/\|r\|$. The thresholds are the 95th percentile of a χ^2 distribution: 5.991 for a 2-DoF monocular residual, 7.815 for a 3-DoF stereo one.

src/slam/optimize/solver.py

```
def _irls(r, stereo, depth, info, active, robust):
    """Per-observation weights, total (robust) cost, and chi2 inlier test."""
    chi2 = info * np.einsum("ki,ki->k", r, r)
    th = np.where(stereo, CHI2_STEREO, CHI2_MONO)
    w, rho = info.copy(), chi2.copy()
    if robust:
        s, d = np.sqrt(chi2), np.sqrt(th)
        big = s > d
        w[big] *= d[big] / s[big]
        rho[big] = 2.0 * d[big] * s[big] - th[big]
    ok = active & (depth > 0)
    w[~ok] = 0.0
    return w, float(rho[ok].sum()), (chi2 < th) & (depth > 0)
```

8.4 Motion-only BA

Points fixed, one pose free: six unknowns. Gauss–Newton on $H \delta = -g$ with $H = \Sigma J^T W J$ and $g = \Sigma J^T W r$. The structure that matters is the outer loop: four rounds, and after each one every observation is re-tested against χ^2 , so an observation wrongly rejected early can come back once the pose has improved. The final round runs without the robust kernel, on inliers only.

src/slam/optimize/solver.py

```
def motion_only_ba(
    T_cw: np.ndarray,
    Xw: np.ndarray,
    z: np.ndarray,
    info: np.ndarray,
    cam: StereoCalib,
    rounds: int = 4,
    iters: int = 10,
```

8.5 Local BA, and the Schur complement

Now both poses and points are free. With N keyframes and P points the system is $(6N + 3P)$ square — for a typical local window, 20 poses and 5000 points, that is 15120×15120 . Solving it directly is hopeless; exploiting its structure is easy. Write the normal equations in blocks:

$$\begin{bmatrix} H_{pp} & H_{px} \\ H_{px}^T & H_{xx} \end{bmatrix} \begin{bmatrix} \delta_p \\ \delta_x \end{bmatrix} = - \begin{bmatrix} g_p \\ g_x \end{bmatrix} \quad (8.10)$$

The key observation: H_{xx} is **block-diagonal with 3×3 blocks**, because no residual involves two different points. Inverting it costs nothing. Eliminating the points gives the *reduced camera system*, of size $6N$ only:

$$S = H_{pp} - H_{px} H_{xx}^{-1} H_{px}^T, \quad S \delta_p = -g_p + H_{px} H_{xx}^{-1} g_x \quad (8.11)$$

and the points follow by back-substitution:

$$\delta_x = H_{xx}^{-1} (-g_x - H_{px}^T \delta_p) \quad (8.12)$$

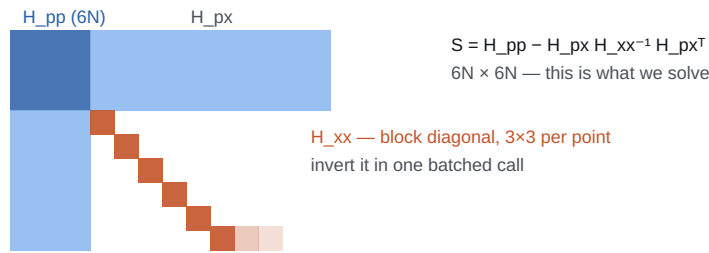


Figure 8.1. Why bundle adjustment is tractable. Points never couple to each other, so the big block is block-diagonal and can be eliminated, leaving a system the size of the poses alone.

src/slam/optimize/solver.py

```
def _schur_step(lin: _Lin, w, obs: Obs, fidx, n_pts: int, lam: float):
    """Solve the damped normal equations, eliminating points via the Schur complement."""
    Jx = lin.J @ lin.R
    wJx = w[:, None, None] * Jx
    Hxx = np.zeros((n_pts, 3, 3))
    gx = np.zeros((n_pts, 3))
    np.add.at(Hxx, obs.point, np.einsum("kai,kaj->kij", wJx, Jx))
    np.add.at(gx, obs.point, np.einsum("kai,ka->ki", wJx, lin.r))
    Hxx[:, _D3, _D3] = Hxx[:, _D3, _D3] * (1.0 + lam) + 1e-9
    Hxx_inv = np.linalg.inv(Hxx)

    nf = int(fidx.max()) + 1
    if nf == 0:
        return np.zeros((0, 6)), -np.einsum("qab,qb->qa", Hxx_inv, gx)

    f = fidx[obs.pose]
    m = f >= 0
    Jp = lin.J[m] @ _dxi(lin.Xc[m])
    wJp = w[m, None, None] * Jp
    Hpp = np.zeros((nf, 6, 6))
    gp = np.zeros((nf, 6))
    np.add.at(Hpp, f[m], np.einsum("kai,kaj->kij", wJp, Jp))
    np.add.at(gp, f[m], np.einsum("kai,ka->ki", wJp, lin.r[m]))
    Hpp[:, _D6, _D6] = Hpp[:, _D6, _D6] * (1.0 + lam) + 1e-9
    A = sp.bsr_matrix((Hpp, np.arange(nf), np.arange(nf + 1)), shape=(6 * nf,) * 2)

    Hpx = np.einsum("kai,kaj->kij", wJp, Jx[m])
    rows = np.broadcast_to(6 * f[m][:, None, None] + _D6[None, :, None], Hpx.shape).ravel()
    cols = np.broadcast_to(3 * obs.point[m][:, None, None] + _D3[None, None, :], Hpx.shape).ravel()
    B = sp.csr_matrix((Hpx.ravel(), (rows, cols)), shape=(6 * nf, 3 * n_pts))
```

```

C = sp.bsr_matrix((Hxx_inv, np.arange(n_pts), np.arange(n_pts + 1)), shape=(3 * n_pts,) * 2)

BC = B @ C
S = A - BC @ B.T
rhs = -gp.ravel() + BC @ gx.ravel()
dp = np.linalg.solve(S.toarray(), rhs) if nf <= 200 else spla.spsolve(S.tocsc(), rhs)
dx = C @ (-gx.ravel() - B.T @ dp)
return dp.reshape(nf, 6), dx.reshape(n_pts, 3)

```

Around that step sits Levenberg–Marquardt. `bundle_adjust(poses, fixed, X, obs, cam, iters=(5, 10), stop=None)` takes the poses with a boolean mask saying which are held fixed, the points, the observations, and an optional abort callback so the tracker can interrupt a long solve. Its loop is the classic damped one — take the step, accept it if the cost went down and relax the damping, otherwise reject it and damp harder:

src/slam/optimize/solver.py

```

for stage, n_iter in enumerate(iters):
    robust = stage < len(iters) - 1
    lam = 1e-4
    w, cost, _ = _cost(lin, obs, active, robust)
    for _ in range(n_iter):
        if stop is not None and stop():
            break
        dp, dx = _schur_step(lin, w, obs, fidx, len(X), lam)
        Ts_new = Ts.copy()
        for k, i in enumerate(free):
            Ts_new[i] = exp_se3(dp[k]) @ Ts[i]
        X_new = X + dx
        lin_new = _linearize(Ts_new, X_new, obs, cam)
        w_new, cost_new, _ = _cost(lin_new, obs, active, robust)
        if cost_new < cost:
            done = cost - cost_new < 1e-6 * cost
            Ts, X, lin, w, cost = Ts_new, X_new, lin_new, w_new, cost_new
            lam = max(lam / 3.0, 1e-8)
            if done:
                break
        else:
            lam *= 10.0
            if lam > 1e6:
                break
    _, _, good = _cost(lin, obs, active, robust=False)
    active &= good
    if stop is not None and stop():
        break
return list(Ts), X, active

```

Two stages, the first with the robust kernel and the second without; observations failing χ^2 after a stage are dropped for good. That is ORB-SLAM's structure exactly.

Why this way

Gauge freedom: shifting every pose and every point by the same rigid transform does not change any residual, so the system is singular unless something is held still. In local BA the fixed set is the keyframes that observe local points but are not themselves being optimised, plus keyframe 0. In the loop-closure pose graph it is the matched keyframe. Forget this and the solver wanders.

Check yourself

1. Why is H_{xx} block-diagonal? What would have to be true of the residuals for it not to be?
2. What does the Huber weight do to an observation with a 40-pixel residual, and why is that better than deleting it outright on the first iteration?
3. Local BA with 20 keyframes and 5000 points: what is the size of the matrix actually factorised?

9

Local mapping: growing and cleaning the map

A keyframe arrives on a queue. This thread turns it into map structure, and it is where most of the compute goes — about 430 ms per keyframe, of which bundle adjustment is 252.

src/slam/mapping.py

```
def process(self, kf: KeyFrame) -> None:
    with self.map.lock:
        self._add_keyframe(kf)
        self._cull_points(kf)
        self._triangulate(kf)
        if not self.queue:
            self._fuse_neighbors(kf)
    self.abort = False
    if not self.queue and len(self.map.keyframes) > 2:
        self._local_ba(kf)
    with self.map.lock:
        self._cull_keyframes(kf)
    if self.loop is not None:
        self.loop.insert(kf)
```

9.1 Cull recent points before trusting them

New points are guilty until proven useful. A point must be found in at least a quarter of the frames that predicted it, and reach three observations within two keyframes of its birth, or it is deleted:

src/slam/mapping.py

```
def _cull_points(self, kf: KeyFrame) -> None:
    keep = []
    for mp in self.recent:
        if mp.bad:
            continue
        age = kf.id - mp.first_kf_id
        if mp.n_found / mp.n_visible < 0.25 or (age >= 2 and mp.n_obs() <= 3):
            mp.set_bad()
        elif age < 3:
            keep.append(mp)
    self.recent = keep
```

9.2 Create new points by triangulation

Only unmatched keypoints are candidates, and only against covisible keyframes with enough baseline. Candidate pairs come from the vocabulary (same node), then the epipolar constraint decides geometry: a point in one image must lie on a line in the other, and the line is given by the fundamental matrix

$$F_{12} = K^{-T} [t_{12}]_{\times} R_{12} K^{-1}, \quad x_1^T F_{12} x_2 = 0 \quad (9.13)$$

src/slam/mapping.py

```
def _epipolar_matches(k1: KeyFrame, k2: KeyFrame) -> tuple[np.ndarray, np.ndarray]:
    """Unmatched keypoint pairs in the same vocabulary node that satisfy the epipolar constraint."""
    f1 = np.flatnonzero(np.equal(k1.points, None))
    f2 = np.flatnonzero(np.equal(k2.points, None))
    if k1.node is None or k2.node is None or not len(f1) or not len(f2):
        return np.zeros(0, int), np.zeros(0, int)
    a, b = same_group_pairs(k1.node[f1], k2.node[f2])
    p, q = f1[a], f2[b]
    d = hamming(k1.descriptors[p], k2.descriptors[q])

    x1, x2 = k1.keypoints[p], k2.keypoints[q]
    line = np.c_[x1, np.ones(len(p))] @ _fundamental(k1, k2)
    num = line[:, 0] * x2[:, 0] + line[:, 1] * x2[:, 1] + line[:, 2]
    s2 = SCALES[k2.octave[q]] ** 2
    epi = num**2 / (line[:, 0]**2 + line[:, 1]**2) < 3.84 * s2
    e = k2.camera.K @ (k2.T_cw[:3, :3] @ k1.center + k2.T_cw[:3, 3])
    far = ((x2 - e[:2] / e[2]) ** 2).sum(axis=1) >= 100 * s2
    stereo = ~np.isnan(k1.u_right[p]) | ~np.isnan(k2.u_right[q])
    ok = (d <= TH_LOW) & epi & (stereo | far)
    p, q, _ = one_to_one(p[ok], q[ok], d[ok], TH_LOW)
    keep = rotation_consistent(k1.angle[p], k2.angle[q])
    return p[keep], q[keep]
```

Surviving pairs are triangulated, but not blindly. The parallax between the two viewing rays is compared against the parallax the stereo rig alone would give; if the rig wins, the stereo unprojection is used instead of the two-view triangulation. Then the candidate must reproject within χ^2 in both keyframes, have positive depth in both, and be scale-consistent with the octaves it was seen at:

src/slam/mapping.py

```
def _create_points(self, k1: KeyFrame, k2: KeyFrame, i1: np.ndarray, i2: np.ndarray) -> None:
    """ORB-SLAM2 CreateNewMapPoints checks: parallax vs stereo, depth, chi2, scale ratio."""
    cam = k1.camera
    c, f = np.array([cam.cx, cam.cy]), np.array([cam.fx, cam.fy])
    xn1, xn2 = (k1.keypoints[i1] - c) / f, (k2.keypoints[i2] - c) / f
    ray1 = np.c_[xn1, np.ones(len(i1))] @ k1.T_wc[:3, :3].T
    ray2 = np.c_[xn2, np.ones(len(i2))] @ k2.T_wc[:3, :3].T
    cos_rays = np.einsum("ij,ij->i", ray1, ray2) / (np.linalg.norm(ray1, axis=1) * np.linalg.norm(ray2, axis=1))
```



```

d1, d2 = stereo_depth(k1)[i1], stereo_depth(k2)[i2]
st1, st2 = d1 > 0, d2 > 0
cps1 = np.where(st1, np.cos(2 * np.arctan2(cam.baseline / 2, np.nan_to_num(d1, nan=1.0))),
cos_rays + 1)
cps2 = np.where(st2, np.cos(2 * np.arctan2(cam.baseline / 2, np.nan_to_num(d2, nan=1.0))),
cos_rays + 1)

dlt = (cos_rays < np.minimum(cps1, cps2)) & (cos_rays > 0) & (st1 | st2 | (cos_rays <
0.9998))

s1 = ~dlt & st1 & (cps1 < cps2)
s2 = ~dlt & ~s1 & st2 & (cps2 < cps1)
X = np.full((len(i1), 3), np.nan)
if dlt.any():
    X[dlt] = triangulate_dlt(k1.T_cw, k2.T_cw, xn1[dlt], xn2[dlt])
if s1.any():
    X[s1] = unproject(k1, i1[s1], k1.T_wc)
if s2.any():
    X[s2] = unproject(k2, i2[s2], k2.T_wc)

ok = np.isfinite(X).all(axis=1)
X = np.nan_to_num(X)
ok &= _reprojection_ok(k1, i1, X) & _reprojection_ok(k2, i2, X)
dist1 = np.linalg.norm(X - k1.center, axis=1)
dist2 = np.linalg.norm(X - k2.center, axis=1)
ratio_dist = dist2 / np.maximum(dist1, 1e-12)
ratio_oct = SCALES[k1.octave[i1]] / SCALES[k2.octave[i2]]
factor = 1.5 * SCALE
ok &= (dist1 > 0) & (dist2 > 0) & (ratio_dist * factor >= ratio_oct) & (ratio_dist <= ra-
tio_oct * factor)

for a in np.flatnonzero(ok):
    mp = self.map.new_point(X[a], k1, i1[a])
    mp.add_observation(k1, i1[a])
    mp.add_observation(k2, i2[a])
    mp.update_descriptor()
    mp.update_normal_and_depth()
    self.recent.append(mp)

```

9.3 Fuse duplicates

The same physical corner is often created twice, from two keyframes that did not yet share a link. Fusion projects each point into a neighbour and merges it with whatever is already at that keypoint, keeping the one with more observations:

src/slam/mapping.py

```

def fuse(kf: KeyFrame, mps: list[MapPoint], th: float = 3.0) -> None:
    """Project points into `kf`; merge with the matched keypoint's point or add an observation."""
    mps = [mp for mp in mps if not mp.bad and kf not in mp.obs]
    if not mps:
        return
    Xw, normal, dmin, dmax, desc = point_arrays(mps)
    pidx, uv, _, level = visible(Xw, normal, dmin, dmax, kf.T_cw, kf.camera)
    if not len(pidx):
        return

    a, j = window_pairs(kf.keypoints, uv[:, 0], uv[:, 1], th * SCALES[level])
    p = pidx[a]
    oct_j = kf.octave[j]
    ok = (oct_j >= level[a] - 1) & (oct_j <= level[a]) & _reprojection_ok(kf, j, Xw[p])
    p, j = p[ok], j[ok]
    g, k, best, _ = best_two(p, hamming(desc[p], kf.descriptors[j]))
    good = best <= TH_LOW
    for pi, ji in zip(g[good], j[k[good]]):
        mp = mps[pi]
        if mp.bad or kf in mp.obs:
            continue
        other = kf.points[ji]
        if other is None or other.bad:
            mp.add_observation(kf, ji)
        elif other is not mp:
            if other.n_obs() > mp.n_obs():
                mp.replace(other)
            else:
                other.replace(mp)

```

9.4 Local bundle adjustment

The window is the new keyframe plus its covisible neighbours; every point they see is optimised with them; and keyframes that see those points but are outside the window are included as *fixed* constraints — they pull, but they do not move. That is what stops the local window from drifting away from the rest of the map.

src/slam/mapping.py

```

def _local_ba(self, kf: KeyFrame) -> None:
    with self.map.lock:
        version = self.map.version
        local = [kf] + kf.best_covisibles()
        for k in local:
            k.ba_local = kf.id
        pts = list({mp: None for k in local for _, mp in k.valid_points()})
        fixed = []

```

```

        for mp in pts:
            for k in mp.obs:
                if k.ba_local != kf.id and k.ba_fixed != kf.id:
                    k.ba_fixed = kf.id
                    fixed.append(k)
            kfs = local + fixed
            kidx = {k: n for n, k in enumerate(kfs)}
            links = [(kidx[k], q, k, i) for q, mp in enumerate(pts) for k, i in mp.obs.items() if k
in kidx]

            if len(links) < 10:
                return
            pose_i, pt_i, obs_kf, kp_i = zip(*links)
            z = np.array([[*k.keypoints[i], k.u_right[i]] for k, i in zip(obs_kf, kp_i)])
            info = np.array([INV_SIGMA2[k.octave[i]] for k, i in zip(obs_kf, kp_i)])
            obs = Obs(np.array(pose_i), np.array(pt_i), z, info)
            is_fixed = np.array([n >= len(local) or k is self.map.origin for n, k in enumerate(kfs)])
            poses = [k.T_cw for k in kfs]
            X = np.stack([mp.Xw for mp in pts])

            new_poses, new_X, inl = bundle_adjust(poses, is_fixed, X, obs, kf.camera, stop=lambda:
self.abort)

            with self.map.lock:
                if self.map.version != version:
                    return # a loop correction landed while we optimized; these poses are stale
                for a in np.flatnonzero(~inl):
                    pts[pt_i[a]].erase_observation(obs_kf[a])
                for k, T, fx in zip(kfs, new_poses, is_fixed):
                    if not fx and not k.bad:
                        k.T_cw = T
                for mp, x in zip(pts, new_X):
                    if not mp.bad:
                        mp.Xw = x
                update_normals_and_depths(pts)

```

Note what happens around the solve: the problem is assembled under the map lock, the solve runs *without* it, and the write-back checks `map.version` and discards the result if a loop closure landed in the meantime. That is Chapter 11's subject.

9.5 Cull redundant keyframes

Finally, any keyframe whose close points are 90% visible in at least three other keyframes at the same or finer scale is redundant and gets removed. This is what keeps the map from growing linearly when the car stops at a light:

src/slam/mapping.py

```

def _cull_keyframes(self, kf: KeyFrame) -> None:
    """Drop keyframes whose close points are 90% seen by >= 3 others at the same or finer
    scale."""
    for k in kf.best_covisibles():
        if k is self.map.origin:
            continue
        depth = stereo_depth(k)
        th = self.th_depth * k.camera.baseline
        n_pts = n_red = 0
        for i, mp in k.valid_points():
            if not depth[i] > 0 or depth[i] > th:
                continue
            n_pts += 1
            if mp.n_obs() > 3:
                level = k.octave[i]
                n = sum(1 for k2, j in mp.obs.items() if k2 is not k and k2.octave[j] <= level +
1)
                n_red += n >= 3
        if n_pts and n_red > 0.9 * n_pts:
            k.set_bad()

```

Check yourself

1. Why triangulate at all, when every close keypoint already has stereo depth?
2. A point is observed by two keyframes 10 cm apart, 60 m away. What does the parallax test do with it, and why is that the right answer?
3. Which of the culling rules keeps memory bounded when the car is stationary?

10

Loop closing: recognising a place and fixing the past

After several kilometres the estimate has drifted by metres. When the car re-enters a street it mapped twenty minutes ago, the system should notice and pull the whole trajectory back into shape. Three things must happen: recognise the place, prove it geometrically, and distribute the correction.

10.1 Recognise: bag of words

Each descriptor is pushed down a vocabulary tree — ten children per node, six levels, trained offline on millions of ORB descriptors — and lands in a leaf, its *word*. A keyframe becomes a sparse vector of word counts weighted by inverse document frequency, L1-normalised:

src/slam/features/vocab.py

```
def transform(self, desc: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Word id and direct-index node id for each descriptor (greedy tree descent)."""
    n = len(desc)
    cur = np.zeros(n, dtype=np.int64)
    node = np.zeros(n, dtype=np.int64)
    ar = np.arange(self.k)
    for level in range(self.L):
        cnt = self.child_count[cur]
        if not cnt.any():
            break
        slots = np.minimum(self.child_start[cur, None] + ar, len(self.children) - 1)
        kids = self.children[slots]
        d = np.where(ar < cnt[:, None], hamming(desc[:, None, :], self.desc[kids]), 1 << 20)
        cur = np.where(cnt > 0, kids[np.arange(n), d.argmin(axis=1)], cur)
        if level + 1 == self.L - LEVELS_UP:
            node = cur.copy()
    return self.word[cur], node
```

Two keyframes are compared with the DBoW2 L1 score, which is 1 for identical vectors and 0 for vectors sharing nothing:

$$s(a, b) = 1 - \frac{1}{2} \sum_i |a_i - b_i| = \frac{1}{2} \sum_i \in \text{both} (|a_i| + |b_i| - |a_i - b_i|) \quad (10.14)$$

src/slam/features/vocab.py

```
def score(a: tuple[np.ndarray, np.ndarray], b: tuple[np.ndarray, np.ndarray]) -> float:
    """DBow2 L1 score in [0, 1]."""
    _, i, j = np.intersect1d(a[0], b[0], assume_unique=True, return_indices=True)
    x, y = a[1][i], b[1][j]
    return float(0.5 * (x + y - np.abs(x - y)).sum())
```

Querying every keyframe would be linear in map size, so an inverted file maps each word to the keyframes containing it, and only keyframes sharing enough words are scored. Scores are then accumulated over covisible groups, so a place seen by a cluster of keyframes beats a single lucky match:

src/slam/loop.py

```
def candidates(self, kf: KeyFrame, min_score: float, exclude: set[KeyFrame]) -> list[KeyFrame]:
    common: dict[KeyFrame, int] = {}
    for w in kf.bow[0]:
        for k in self.inv.get(int(w), ()):
            if not k.bad and k not in exclude:
                common[k] = common.get(k, 0) + 1
    if not common:
        return []
    th = 0.8 * max(common.values())
    scores = {k: s for k, n in common.items() if n > th and (s := score(kf.bow, k.bow)) >=
min_score}
    # Accumulate over covisible neighbours so a place seen by a cluster of KFs wins.
    acc: dict[KeyFrame, float] = {}
    for k, s in scores.items():
        best, total = k, s
        for n in k.best_covisibles(10):
            if n in scores:
                total += scores[n]
                if scores[n] > scores[best]:
                    best = n
        acc[best] = max(acc.get(best, 0.0), total)
    if not acc:
        return []
    top = max(acc.values())
    return [k for k, t in acc.items() if t > 0.75 * top]
```

Appearance alone is not enough — brick walls repeat. A candidate must stay consistent across three consecutive keyframes before it is even considered:

src/slam/loop.py

```

def _detect(self, kf: KeyFrame) -> list[KeyFrame]:
    """Candidates that stayed consistent (covisible groups overlap) for 3 consecutive key-
    frames."""
    if kf.id < self.last_loop + 10:
        return []
    cov = kf.best_covisibles()
    min_score = min((score(kf.bow, c.bow) for c in cov), default=0.0)
    cands = self.db.candidates(kf, min_score, set(cov) | {kf})
    groups, enough = [], []
    for c in cands:
        group = set(c.best_covisibles()) | {c}
        consistent = False
        for prev, count in self.groups:
            if group & prev:
                consistent = True
                groups.append((group, count + 1))
                if count + 1 >= CONSISTENCY and c not in enough:
                    enough.append(c)
        if not consistent:
            groups.append((group, 0))
    self.groups = groups
    return enough

```

10.2 Prove: geometry

Both keyframes have 3-D points. Take matches between them, and find the rigid transform aligning the two point sets with Horn's closed-form solution inside a RANSAC loop; an inlier must reproject within χ^2 in *both* images. Twenty inliers are required.

src/slam/loop.py

```

def _ransac_se3(k1: KeyFrame, k2: KeyFrame, i1: np.ndarray, i2: np.ndarray, iters: int = 300):
    """T_c1c2 from 3D-3D Horn hypotheses; inliers reproject within chi2(2, 99%) in both views."""
    cam = k1.camera
    P1 = np.stack([k1.points[i].Xw for i in i1]) @ k1.T_cw[:3, :3].T + k1.T_cw[:3, 3]
    P2 = np.stack([k2.points[j].Xw for j in i2]) @ k2.T_cw[:3, :3].T + k2.T_cw[:3, 3]
    th1 = 9.21 * SCALES[k1.octave[i1]] ** 2
    th2 = 9.21 * SCALES[k2.octave[i2]] ** 2

    def inliers(T: np.ndarray) -> np.ndarray:
        Q1 = P2 @ T[:3, :3].T + T[:3, 3]
        Ti = invert(T)
        Q2 = P1 @ Ti[:3, :3].T + Ti[:3, 3]
        e1 = ((cam.project(Q1) - k1.keypoints[i1]) ** 2).sum(axis=1)
        e2 = ((cam.project(Q2) - k2.keypoints[i2]) ** 2).sum(axis=1)
        return (Q1[:, 2] > 0) & (Q2[:, 2] > 0) & (e1 < th1) & (e2 < th2)

    rng = np.random.default_rng(0)
    best = np.zeros(len(i1), dtype=bool)
    for _ in range(iters):
        s = rng.choice(len(i1), 3, replace=False)
        inl = inliers(horn_se3(P2[s], P1[s]))
        if inl.sum() > best.sum():
            best = inl
    if best.sum() < 20:
        return None
    T = horn_se3(P2[best], P1[best])
    inl = inliers(T)
    return (T, inl) if inl.sum() >= 20 else None

```

That transform gives a corrected pose for the current keyframe. All points of the loop side are then projected into it to gather many more matches, and a final motion-only BA at that corrected pose must keep at least 40 inliers. Only then is the loop accepted:

src/slam/loop.py

```

def _verify(self, kf: KeyFrame, cand: list[KeyFrame]):
    for c in cand:
        i1, i2 = _bow_matches(kf, c)
        if len(i1) < 20:
            continue
        ransac = _ransac_se3(kf, c, i1, i2)
        if ransac is None:
            continue
        T12, inl = ransac
        T = T12 @ c.T_cw
        matched = np.full(len(kf.keypoints), None, dtype=object)
        matched[i1[inl]] = c.points[i2[inl]]
        loop_pts = list({mp: None for k in [c] + c.best_covisibles() for _, mp in k.valid_
            id_points()})
        _search_loop(kf, T, loop_pts, matched)
        idx = np.flatnonzero(np.not_equal(matched, None))
        X = np.stack([mp.Xw for mp in matched[idx]])
        z = np.column_stack([kf.keypoints[idx], kf.u_right[idx]])
        T, good = motion_only_ba(T, X, z, INV_SIGMA2[kf.octave[idx]], kf.camera)
        if good.sum() >= 40:
            matched[idx[~good]] = None
            return c, T, matched, loop_pts
    return None

```

10.3 Fix: correct the neighbourhood, then the graph

The correction is applied in two stages. First locally and immediately: the current keyframe and its covisible neighbours are moved to the corrected pose, their points move with them, and duplicated points across the seam are fused. Then globally: an essential-graph pose-graph optimisation distributes the remaining error over the whole trajectory.

src/slam/loop.py

```

def _correct(self, kf: KeyFrame, match: KeyFrame, T_kf: np.ndarray, matched: np.ndarray,
    loop_pts: list[MapPoint]) -> None:
    self.map.version += 1
    kf.update_connections()
    connected = [kf] + kf.best_covisibles()
    old = {k: k.T_cw for k in connected} # only these move before the pose graph runs
    corrected = {k: (k.T_cw @ kf.T_wc) @ T_kf for k in connected}

    # Move connected keyframes and their points with the loop correction.
    moved: set[MapPoint] = set()
    for k in connected:
        corr = invert(corrected[k]) @ k.T_cw
        for _, mp in k.valid_points():

```

```

        if mp not in moved:
            moved.add(mp)
            mp.Xw = corr[:3, :3] @ mp.Xw + corr[:3, 3]
        k.T_cw = corrected[k]
    for mp in moved:
        mp.update_normal_and_depth()

    # Fuse duplicated points on both sides of the loop.
    for i in np.flatnonzero(np.not_equal(matched, None)):
        lp, cur = matched[i], kf.points[i]
        if lp.bad:
            continue
        if cur is None or cur.bad:
            lp.add_observation(kf, i)
        elif cur is not lp:
            cur.replace(lp)
    before = {k: set(k.best_covisibles()) for k in connected}
    for k in connected:
        fuse(k, loop_pts, th=4.0)
    new_links = {}
    for k in connected:
        k.update_connections()
        new_links[k] = set(k.best_covisibles()) - before[k] - set(connected)

    self._optimize_essential_graph(match, old, new_links)
    match.loop_edges.add(kf)
    kf.loop_edges.add(match)
    self.last_loop = kf.id
    self.loops.append((kf.frame_id, match.frame_id))

```

The pose graph minimises, over all keyframe poses, the disagreement of each edge with its measured relative pose:

$$r_{ij} = \log(\hat{T}_{ji}^{-1} T_j T_i^{-1}) \in \mathbb{R}^6 \quad (10.15)$$

where $\hat{T}_{ji}$ is what the relative pose was *before* the loop correction for ordinary edges, and the corrected value for the new loop links. Points are not in this optimisation at all — each one is carried along by its reference keyframe afterwards. That is what makes it fast enough to run on 1500 keyframes.

src/slam/loop.py

```

def _optimize_essential_graph(self, match: KeyFrame, old: dict, new_links: dict) -> None:
    """Spanning tree + strong covisibility + loop edges, measured with pre-correction poses;
    new loop links use corrected poses. The matched keyframe fixes the gauge."""
    kfs = sorted(self.map.keyframes.values(), key=lambda k: k.id)
    idx = {k: n for n, k in enumerate(kfs)}
    edges: dict[tuple[int, int], np.ndarray] = {}

    def add(a: KeyFrame, b: KeyFrame, Ta: np.ndarray, Tb: np.ndarray) -> None:
        if a in idx and b in idx and a is not b:
            key = (idx[a], idx[b]) if idx[a] < idx[b] else (idx[b], idx[a])
            if key not in edges:
                i, j = key
                Ti, Tj = (Ta, Tb) if idx[a] == i else (Tb, Ta)
                edges[key] = Tj @ invert(Ti)

    # Loop links first: after fusion the same pairs are also strongly covisible, and a
    # covisibility edge measured with drifted poses would otherwise encode the drift.
    for k, links in new_links.items():
        for n in links:
            add(k, n, k.T_cw, n.T_cw)
    for k in kfs:
        o = old.get(k, k.T_cw)
        if k.parent is not None and not k.parent.bad:
            add(k.parent, k, old.get(k.parent, k.parent.T_cw), o)
        for n in k.loop_edges:
            add(n, k, old.get(n, n.T_cw), o)
        for n, w in k.covis.items():
            if w >= MIN_COVIS_EDGE:
                add(n, k, old.get(n, n.T_cw), o)

    before = [k.T_cw for k in kfs]
    after = pose_graph_se3(before, [(i, j, T) for (i, j), T in edges.items()],
fixed=(idx[match],))
    corr = {k: invert(Ta) @ Tb for k, Ta, Tb in zip(kfs, after, before)}
    for mp in self.map.points:
        ref = mp.ref_kf if mp.ref_kf in corr else next((k for k in mp.obs if k in corr), None)
        if ref is not None:
            C = corr[ref]
            mp.Xw = C[:3, :3] @ mp.Xw + C[:3, 3]
            mp.update_normal_and_depth()
    for k, T in zip(kfs, after):
        k.T_cw = T
    self.map.version += 1

```

Bug story — the pose graph fighting its own correction

Edges are deduplicated by keyframe pair. After fusion, the two sides of the loop are not only linked by the new loop edges — they are now *strongly covisible*, so the ordinary covisibility rule proposes the same pairs, measured with the old drifted poses. Whichever is inserted first wins.

Inserting covisibility edges first meant the pose graph was told “these two keyframes are 5 m apart” by the drift and “they are here, together” by the loop, and it split the difference. ATE 5.23 m. Inserting loop links first: **0.96 m**. Same code otherwise. ORB-SLAM inserts loop connections first for exactly this reason — and that ordering is worth a comment in any implementation, because nothing crashes when it is wrong.

Check yourself

1. Why require three consecutive consistent detections instead of trusting one high score?
2. Why is the matched (old) keyframe the fixed one in the pose graph, and not keyframe 0?
3. The loop closes. Why does the trajectory printed at the end of the run change for frames from twenty minutes ago, even though those frames were never revisited?

11

Three threads and one lock

Tracking must keep up with the camera. Mapping cannot. That is the whole reason for the threading, and it is where more than half of this system's speed came from.

Local mapping needs about 430 ms per keyframe and a keyframe arrives every three frames, so running it inline caps the system at roughly 1 fps. Moving it to a worker thread lets bundle adjustment — which is NumPy and SciPy, releasing the GIL for the large operations — overlap with the next frames' tracking. Same sequence, same work: **1.1 fps to 6.4 fps**.

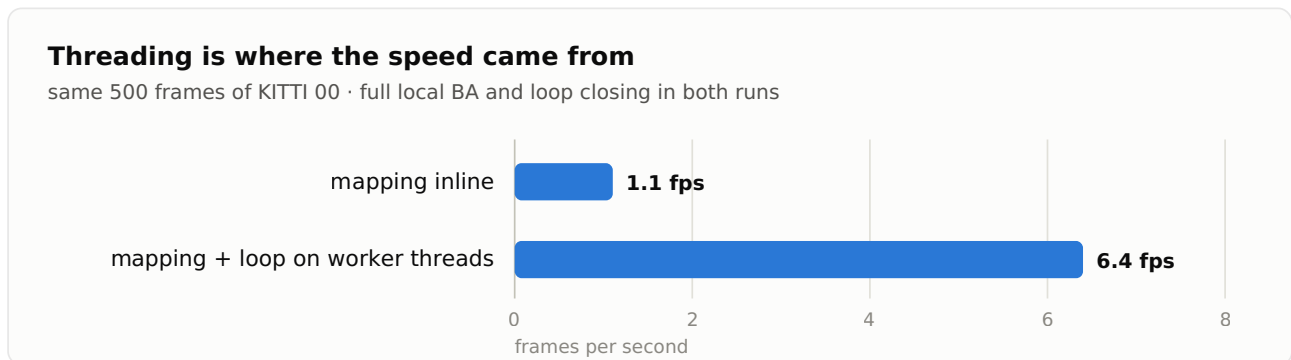


Figure 11.1. Measured on the same 500 frames with identical settings.

Both workers share one small base class — a queue, a condition variable, and an inline mode that keeps tests deterministic. The worker loop is the whole of it:

src/slam/worker.py

```
def _run(self) -> None:
    while True:
        with self._cv:
            while not self.queue and not self._stop:
                self._cv.wait()
            if not self.queue:
                return
            kf = self.queue.popleft()
            self.busy = True
        try:
            self.process(kf)
        except Exception:
            traceback.print_exc() # a dead worker would silently stall the map
        finally:
            self.busy = False
```

An exception inside process is printed rather than allowed to kill the thread: a dead mapping thread does not crash anything, it just silently stops the map growing, which is far harder to notice. In inline mode (`--sync`) the same queue is drained on the calling thread, so tests and debugging runs are deterministic.

11.1 What the lock protects

The map is a graph of Python objects mutated by three threads. The rules we hold to:

- Tracking holds `map.lock` for its whole frame — it walks observation dictionaries, and those must not change underneath it.
- Local mapping holds it while mutating structure, releases it for the BA solve, and re-acquires it to write results back.
- Loop closing holds it for the entire correction, because a half-corrected map is not a map.

Bug story — bundle adjustment undoing a loop closure

Because local BA deliberately releases the lock to solve, a loop correction can land while it is solving. Its snapshot is then stale, and writing it back silently drags the just-corrected keyframes back to where they were. The fix is a version counter on the map: loop correction bumps it, and BA discards its result if the version changed. ORB-SLAM solves the same problem with an explicit stop-request handshake; the counter is the smaller thing that cannot deadlock.

11.2 Where the time actually goes

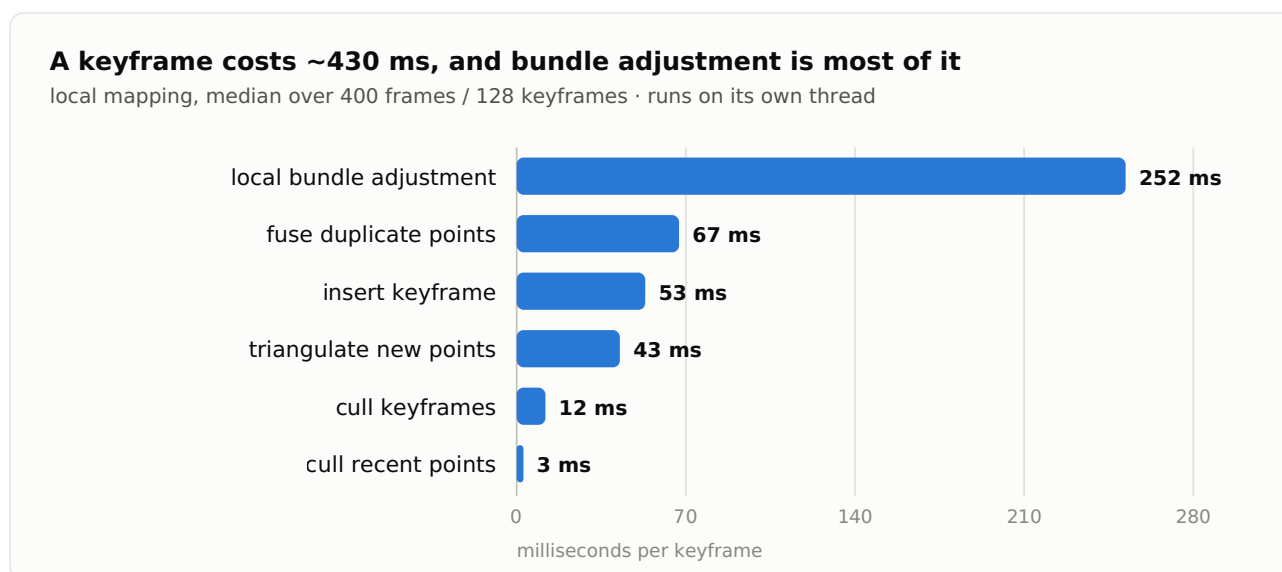


Figure 11.2. Per keyframe, measured over 400 frames. Tracking itself is ~40 ms per frame and feature extraction ~43 ms; the tracking thread stalls when it waits for the map lock.

Two optimisations were tried and *rejected* on measurement, which is worth as much as the two that were kept:

CHANGE	RESULT	KEPT
Mapping + loop closing on worker threads	1.1 → 6.4 fps	yes
Left/right ORB extraction in parallel	51 → 43 ms per frame	yes
Dense Schur complement instead of sparse	2408 → 2715 ms on real problems	no
Cap local BA at 12 covisible keyframes	no speed-up; ATE 1.73 → 3.33 m	no

12

Measuring it: ATE, RPE, and what the numbers mean

Two metrics, answering different questions. Quote the right one for the question you are asked.

12.1 Absolute trajectory error

Align the estimated trajectory to ground truth with a single rigid transform — Horn again — then take the RMS distance between corresponding camera centres. It answers: *how far is the whole trajectory from the truth, as a shape?* It is dominated by the largest global deformation, which is exactly what loop closure fixes.

src/slam/eval.py

```
def ate_rmse(traj_cw: list[np.ndarray], gt_cw: list[np.ndarray]) -> float:
    n = min(len(traj_cw), len(gt_cw))
    est, gt = centers(traj_cw[:n]), centers(gt_cw[:n])
    T = horn_se3(est, gt)
    aligned = est @ T[:3, :3].T + T[:3, 3]
    return float(np.sqrt(np.mean(np.sum((aligned - gt) ** 2, axis=1))))
```

12.2 Relative pose error, KITTI style

Take every sub-path of length 100, 200, ... 800 m, compare the relative motion over it with ground truth, and average. It answers: *how much does the system drift per unit distance?* A system with a perfect loop closure and terrible odometry can have good ATE and bad RPE.

src/slam/eval.py

```
def kitti_rpe(
    traj_cw: list[np.ndarray],
    gt_cw: list[np.ndarray],
    lengths: tuple[int, ...] = (100, 200, 300, 400, 500, 600, 700, 800),
    step: int = 10,
```

12.3 Our numbers

METRIC	THIS SYSTEM	ORB-SLAM2 (C++)
ATE, KITTI 00	1.73 m	1.30 m
Translation drift	0.88 %	0.70 %
Rotation drift	0.32 °/100 m	0.25 °/100 m
Lost frames	0	—
Speed	6 fps	~30 fps

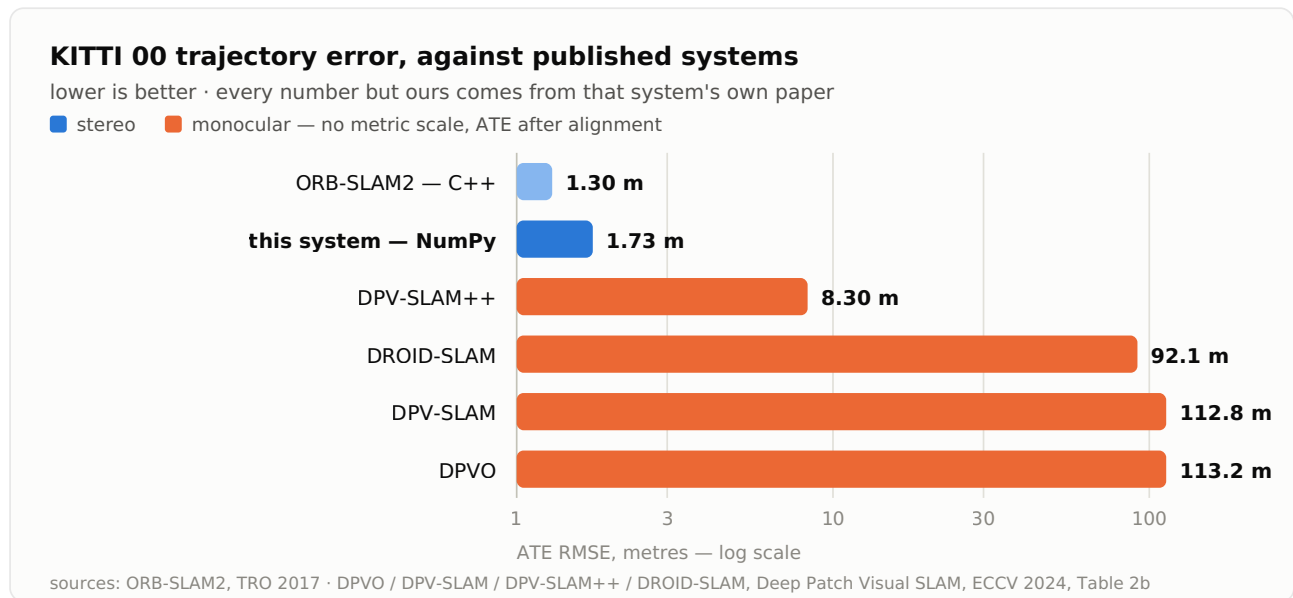


Figure 12.1. Where this sits. We are behind ORB-SLAM2 — say so plainly — and ahead of published learned monocular systems, which is largely because they are monocular and have no metric scale.

Be precise when you present this. The honest claim is not “we beat DROID-SLAM”. It is: a stereo system implemented in NumPy lands in the classical-stereo accuracy band on a 3.7 km sequence, within 33% of the C++ reference, and the learned monocular systems that publish KITTI 00 numbers are an order of magnitude behind on this sequence because scale is unobservable to them.

13

Three bugs worth knowing by heart

These are the stories to tell when someone asks what you learned. Each one has a symptom, a wrong first hypothesis, and a method that found the real cause.

13.1 The dataset was not what it claimed to be

Symptom. Tracking diverged around frame 40, every time, on every variant of the code.

Wrong hypotheses, in order. Bad local BA; bad triangulation; bad loop closure. All were disproved by ablation: the system diverged identically with local BA disabled, with triangulation disabled, and with fusion disabled.

Method that worked. Ask whether the *data* is consistent with itself. Estimate rotation between consecutive images using only the essential matrix — no depth, no map, nothing but the intrinsics — and compare with the rotation implied by the ground-truth poses. They disagreed by 2.7° per frame. Then correlate the image-derived yaw-rate profile against every ground-truth file in the dataset: frames 0–1100 matched *sequence 01*, not sequence 00.

Cause. Half the images came from a mislabelled mirror of the dataset. Every "SLAM bug" for days was the system honestly tracking a different drive.

Lesson, now encoded as a test. `tests/test_kitti_integrity.py` verifies that image motion agrees with ground truth before any SLAM code is blamed.

13.2 Poses drifting off the manifold

Symptom. After the data was fixed: smooth tracking for ~ 35 frames, then error growing geometrically — 0.07 m, 0.35 m, 0.75 m, 1.7 m, 4.3 m — with hundreds of healthy inliers the whole time.

Method. Instrument every boundary and ask which invariant breaks first. Measuring $\|R^T R - I\|$ per frame showed it tripling every frame from round-off at frame 5 to 5×10^{-2} by frame 37.

Cause and fix. The feedback loop in Chapter 3: velocity as a twist, and orthonormalisation inside the estimators.

13.3 The pose graph fighting its own loop correction

Symptom. Loop detection was correct — all closures at genuine revisits — yet ATE got *worse* after closing them (5.23 m), with 0.75 m steps at the loop seams.

Method. Separate detection from correction. Verify against ground truth that each detected pair really is the same place (it was, to within 5 m), then look at where in the trajectory the error entered: not at the loop, but spread over the older half of the map — a correction being distributed wrongly.

Cause and fix. Edge insertion order in the essential graph, Chapter 10. ATE 5.23 $\rightarrow$ 0.96 m.

The transferable lesson

All three were found the same way: form one hypothesis, design a measurement that can *disprove* it, and only then change code. Ablations (turn one component off) and invariant checks (measure something that must be true) beat reading code and guessing, every time.

14

What is missing, and what I would build next

- **Relocalization.** If tracking is lost, the system currently keeps predicting with the velocity model. The right answer is the ORB-SLAM one: query the bag-of-words database for candidate keyframes and solve PnP with RANSAC. The vocabulary and database are already here — this is perhaps 80 lines.
- **Atlas.** If relocalization fails for long enough, freeze the map and start a new one, then merge when the two are recognised as the same place. This is ORB-SLAM3's main structural contribution.
- **Full BA after loop closure.** ORB-SLAM2 runs a global bundle adjustment on a fourth thread after the pose graph. Our loop seams still show ~ 0.7 m steps, which is where that would help.
- **Speed.** At 6 fps the bottleneck is no longer feature extraction but the map lock — mapping holds it ~ 175 ms per keyframe while tracking waits. Shortening those critical sections, or moving the hottest Python loops into numba, is the next honest step toward real time.
- **Learned features.** SuperPoint + LightGlue behind the existing extractor/matcher interface; published results put SuperPoint-based ORB-SLAM3 variants well ahead on hard lighting.

Appendix A — file map

FILE	RESPONSIBILITY
<code>data/frame.py</code>	Frame container; StereoCalib: projection, unprojection, disparity → depth.
<code>data/kitti.py</code>	KITTI sequence reader: images, calibration, ground-truth poses.
<code>geometry/lie.py</code>	SO(3)/SE(3) exp, log, inverse, SO(3) projection.
<code>geometry/align.py</code>	Horn's closed-form rigid alignment.
<code>geometry/triangulate.py</code>	Batched DLT triangulation.
<code>features/orb.py</code>	ORB extraction, grid bucketing, stereo matching, SAD subpixel.
<code>features/matcher.py</code>	Hamming, ragged/window/node candidate generation, projection search, rotation check.
<code>features/vocab.py</code>	DBoW2 vocabulary: tree descent, TF-IDF, L1 score.
<code>map/structure.py</code>	MapPoint, KeyFrame, covisibility, spanning tree, Map, frustum test.
<code>optimize/solver.py</code>	Motion-only BA; sparse Schur-complement bundle adjustment.
<code>optimize/pose_graph.py</code>	Sparse SE(3) pose-graph optimisation.
<code>tracking.py</code>	Per-frame pose: predict, search, optimise, keyframe decision.
<code>mapping.py</code>	Local mapping thread: cull, triangulate, fuse, local BA, keyframe culling.
<code>loop.py</code>	Loop closing thread: BoW database, verification, correction, essential graph.
<code>worker.py</code>	Queue + thread base class shared by mapping and loop closing.
<code>eval.py</code>	ATE, KITTI RPE, KITTI-format trajectory output.
<code>viz.py</code>	Rerun logging: image, features, map, keyframes, trajectories.

Appendix B — glossary

TERM	MEANING
ATE	Absolute trajectory error: RMS distance to ground truth after one rigid alignment.
Bag of words	An image summarised as a sparse histogram of quantised descriptors, for appearance-based place recognition.
Bundle adjustment	Joint non-linear least squares over camera poses and 3-D points, minimising reprojection error.
Covisibility graph	Keyframes linked when they observe the same points; weight = number shared.
Essential graph	Spanning tree + strong covisibility edges + loop edges; the sparse graph used for pose-graph optimisation.
Gauge freedom	The global transform to which the whole solution is invariant; removed by fixing at least one pose.
Huber kernel	Robust cost that grows linearly past a threshold, limiting the influence of outliers.
Keyframe	A frame kept permanently, carrying map structure; intermediate frames only localise.
Motion-only BA	Bundle adjustment with points fixed — solves for one camera pose.
Octave	Pyramid level a feature was detected at; sets its measurement uncertainty.
Parallax	Angle between the two rays seeing a point; small parallax means poorly determined depth.
RPE	Relative pose error: drift measured over fixed-length sub-paths.
Schur complement	Eliminating the point block of the normal equations to leave a system the size of the poses.
SE(3) / SO(3)	The groups of rigid motions / rotations in 3-D.
Twist	The six-vector (ω , v) parameterising a rigid motion in the Lie algebra.

Built from the repository with `scripts/build_notes.py`; every code listing is extracted from `src/slam/` at build time.